

Driver Drowsiness Detection

Jakkaphat Jumratboonsom

Faculty of ICT, Mahidol University, Nakhon Pathom,
Thailand 73170
jakkaphat.jum@student.mahidol.edu

Vichayuth Ngamsittipong

Faculty of ICT, Mahidol University, Nakhon Pathom,
Thailand 73170
vichayuth.nga@student.mahidol.edu

Abstract

Drowsy drivers are a primary cause of many road accidents across the globe. This project presents a real-time drowsy driver detection system based on computer vision and deep learning methodologies. The system detects a driver's drowsiness or alertness using an ensemble of object detection and classification models. The complete system first detects the driver's face and then predicts drowsiness in a live video stream to test actual conditions.

Keywords

Drowsiness detection, face detection, deep learning, computer vision, YOLO, ResNet50, MobileNetV3

I. INTRODUCTION

Fatigued drivers are more likely to have accidents, have slower reaction times, and make poorer decisions. This project addresses this problem by developing a system to detect signs of drowsiness through real-time facial analysis. The core contributions include:

- Implementing and evaluating multiple deep learning models for drowsiness classification.
- Comparing different methods for eye and face detection.
- Evaluating the suggested solution's viability in real-time under real-world conditions.

II. METHODOLOGY

A. Dataset

We used the Driver Drowsiness Detection Dataset (DDD) available on Kaggle website[1]. It includes labeled images of drowsy and non-drowsy drivers under various conditions.

Class distribution is relatively balanced, with thousands of images per class, ensuring sufficient data for training.

To improve robustness, we applied the following image transformation using PyTorch[2]:

- Resize to (224, 224) pixels
- Random horizontal flipping (p=0.5) to generalize for left/right orientations
- RandomRotation(10 degrees)
- ColorJitter (brightness=0.4, contrast=0.4, saturation=0.4, hue=0.2)
- RandomAffine(degree=5, translate=(0.1, 0.1))
- GaussianBlur(kernel_size=3, sigma=(0.1, 0.1))
- Normalize with ImageNet mean and std

These transformations ensure the model is less sensitive to lighting, positioning, and noise.

B. Model Development

To build an effective drowsiness classifier, we experimented with multiple models:

- ResNet50[3] and MobileNetV3[4] from torchvision.models[5], with custom modifications.

- Custom CNN[6] architecture.
- A hybrid CNN + ResNet50 approach for better feature extraction.
- A CNN + MobileNetV3 hybrid strategy for improved feature extraction

To address overfitting/underfitting, we:

- Applied dropout layers and data augmentation
- Tuning learning rates and optimizer settings
- Used early stopping based on validation loss trends[7]

We tracked convergence using training and validation loss curves across epochs. Learning curves clearly indicated whether the models were overfitting or underfitting. For example, when MobileNetV3 showed minimal gap between train and validation accuracy, it indicated good generalization, whereas custom CNN exhibited overfitting.

For object detection, we started with:

- haarcascade_eye.xml[8] for eye detection.
- harrcascade_frontalface_default.xml[9] for face detection.

Our initial focus was to separate the driver's eyes and detect them individually. This approach mostly worked; however, it was rather challenging when only one eye was visible, when there was some reflection, or when the light conditions were different. So, we firstly tried to detect both eyes at the same time and, at a later stage, try to have a more accurate picture of the surroundings, which also failed.

With ballistic face detection and recognition using camera systems and multiple lenses, we worked with face detection using YOLOv8[10]. However, it turned out that YOLOv8n does not actually detect faces, but rather people through bounding box regression for faces alongside other body parts. The problem here is that in some cases, besides faces, YOLO places rectangles around other parts of the body – arms, shoulders – that should not be treated as independent identities. Even with these problems, YOLO seems to be able to perform simultaneous detection of different objects quite efficiently.

This implies that although YOLO cannot be used straight away for face detection since the performance face detection quality is below average, there might be prospects of refining the device to operate solely on face detection and provide more accurate results instantaneously, which would enhance the performance of the system in the future

III. RESULTS

A. Experimental Setup

We used a live demo setup with a webcam to track and forecast the driver's condition in real time in order to validate the model. The objective was to validate the model's capacity to:

- Identify the face in the picture with accuracy

- Correctly classify whether the driver was drowsy or not.

B. Performance Metrics

For the techniques that we used to evaluate performance:

- Confusion Matrix (via sklearn.metrics)[11]
- Accuracy, Precision, Recall, and F1-score

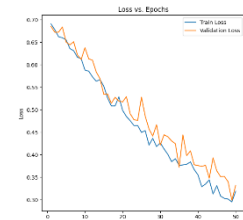
We use confusion_matrix from sklearn.metrics to evaluate our classification performance. This made it easier to comprehend false positives, false negatives and true negatives—all of which are essential for safety-critical systems like this.

C. Model Comparison

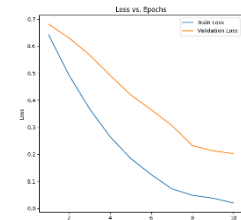
After we try to run and develop model for predict the drowsiness of the driver, this is the table of the comparison which we will compare with accuracy and F1 score.

Model	Accuracy	F1-score
CNN	~78%	~0.76
ResNet50	~95%	~0.93
MobileNetV3	~96%	~0.95
CNN + ResNet50	~97	~0.96
CNN + MobileNetV3	~99	~0.98

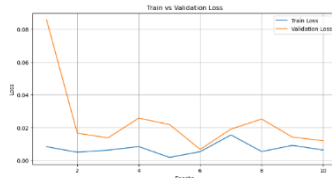
We compared five models in terms of accuracy and F1-score. The custom CNN was the simplest model, offering decent performance but showing signs of overfitting. MobileNetV3 provided a good balance between efficiency and accuracy, performing well in both training and validation phases. ResNet50, being deeper and more powerful, extracted more complex features and achieved high performance. The hybrid CNN + ResNet50 model combined the strengths of both architectures and showed strong results. Ultimately, the hybrid CNN + MobileNetV3 model outperformed others slightly and was selected as the final model used in the real-time demonstration due to its best balance of speed and accuracy.



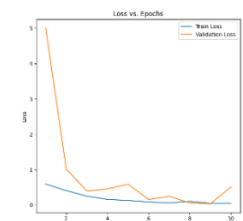
Custom CNN Training and Validation losses



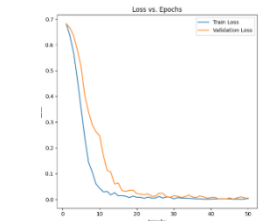
MobileNetV3 Training and Validation losses



ResNet50 Training and Validation losses



CNN + ResNet50 Training and Validation losses



CNN + MobileNetV3 Training and Validation losses

For Face Detection:

- haarcascade_eye.xml: Many false positive and often failed in real-time conditions.
- haarcascade_frontface_default.xml: Better than eye detection but there are some false positive and failed in real-time conditions.
- YOLOv8n: Initially seemed promising, but we found that it detects general person regions not specially the face. This led to incorrect detections when the model highlighted unrated body parts. Despite that, YOLO's architecture allows for further training, and in the future, it can potentially be improved to detect only faces with custom training.

IV. CONCLUSIONS

Our system proves that real-time driver drowsiness detection using deep learning is not only feasible but practical. Among the evaluated models, the hybrid CNN + MobileNetV3 achieved the highest accuracy and F1-score and was used in our final demo.

The object detection component, while functional, highlighted limitations of pre-trained YOLO models for face detection. This motivates future customization.

Future Direction:

- Fine-tune YOLO to detect only facial regions instead of full bodies.
- Integrate temporal features (e.g., eye closure duration or yawning frequency) to improve detection robustness.
- Improve system UX/UI and deploy

REFERENCES

- [1] Driver Drowsiness Dataset (DDD), Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/ismailnasri20/driver-drowsiness-dataset-ddd>
- [2] TORCHVISION.TRANSFORMS, Pytorch. [Online]. Available: <https://pytorch.org/vision/0.9/transforms.html>
- [3] resnet50, Pytorch. [Online]. Available: <https://pytorch.org/vision/main/models/generated/torchvision.models.resnet50.html>
- [4] MobileNetV3, Pytorch. [Online]. Available: <https://pytorch.org/vision/main/models/mobilenetv3.html>
- [5] TORCHVISION.MODELS, Pytorch. [Online]. Available: <https://pytorch.org/vision/0.9/models.html>
- [6] Introduction to Convolution Neural Network, GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/introduction-convolution-neural-network/>
- [7] Regularization by Early Stopping, GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/regularization-by-early-stopping/>
- [8] Haar-cascade-files, Austinjoyal. [Online]. Available: https://github.com/austinjoyal/haar-cascade-files/blob/master/haarcascade_eye.xml
- [9] Opencv, kipi. [Online]. Available: https://github.com/kipi/opencv/blob/master/data/haarcascades/haarcascade_frontalface_default.xml
- [10] Explore Ultralytics YOLOv8, Ultralytics. [Online]. Available: <https://yolov8.com>
- [11] Confusion_matrix, scikit-learn. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html